EPSR-test-ex81 dockato: fixed dihedral angle (DIH groups)

Program(s): dockato | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

`dockato` [1] builds larger molecules from `.ato` building blocks: copies of the first molecule of a docking file are bonded onto free sites of a docked file, removing one atom of each to form the new bond. With dminrr = dmaxrr the new bond gets a fixed dihedral and a DIH rotational group instead of a ROT group.

Code exercised

Fortran (original, fortran_src/):

- `dockato.f`: `dockato` (driver, parameters from command line or stdin), `rotatem`, `random_rotatem`, `search_unsaturated`, `find_valence`
- `dockato.f`: `test_overlap`, `bond_molecule` (ROT/DIH groups), `replace_references`, `add_neighbours`, `remove_atoms`, `clean_atom_types`
- `ato_manipulation_rev4.f`: `get_direct_bonds`, `get_rotgrp`, `centreofmass`, `ftype`; `ato_read_and_save.f`: `moloutc`

Rust (port, rust/epsr-core/src/):

- `tools/dockato.rs`: `run`, `rotatem`, `random_rotatem`, `search_unsaturated`, `find_valence`, `test_overlap`
- `tools/dockato.rs`: `bond_molecule`, `replace_references`, `add_neighbours`, `remove_atoms`, `clean_atom_types`, `get_direct_bonds`, `get_rotgrp`
- `ato.rs`: `State::moloutc` (ROT/DIH groups), `State::ftype`; `rng.rs`: `Ran1::ran1`

Physical system

The building block `WaterMolecule.ato` of EPSR-test: one water molecule (O1, H1, H1; O-H 0.976 Å, H-H 1.54 Å, SPC/E Lennard-Jones and charges [2]) in a 14.42 Å box at 300 K. Atom `atomd` of the new molecule is placed on `atomv`, the molecule is rotated so that its `atomdr` neighbour lies on the `atomvr` neighbour of `atomv` (equal bond lengths required), an optional second (dihedral) rotation about the new bond uses `atomdrr`/`atomvrr` (random angle in [dminrr, dmaxrr] from `ran1`, the Park-Miller generator with Bays-Durham shuffle [3,4], or a fixed dihedral that creates a DIH group when dminrr = dmaxrr), overlaps are tested with a distance factor, `atomd` and `atomvr` are removed and the new rotational group, bonds and atom types are updated. As in ex80 the water units are joined by O-O bonds (docking H1 on docked O1); the dihedral reference atoms are `H1` on both sides.

Inputs

All parameters on the command line: run `1 ... 4 10 WaterMolecule.ato O1 H1 H1 120 120 yes waterdih` (fixed 120°, overlaps allowed, 4 molecules); run 2 fixed 75° with overlap factor 0.8, 4 molecules (`waterdih2`).

How the accuracy was verified

`commands.txt` (2 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.

Each run writes one new `.ato` file (coordinates, bonds, rotational groups, atom types and the random-number state), compared byte for byte with the stored Fortran output. `waterdih.ato`: 7 atoms (all 4 dockings) and 4 DIH groups; `waterdih2.ato`: 5 atoms and 2 DIH groups, i.e. 2 of 4 placements accepted by the 0.8 overlap test. The case exercises the DIH group output of `moloutc` (DIH writing was added to the port's `moloutc` for this).

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).

[2] H. J. C. Berendsen, J. R. Grigera and T. P. Straatsma, "The missing term in effective pair potentials", J. Phys. Chem. 91, 6269-6271 (1987).

[3] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find", Commun. ACM 31, 1192-1201 (1988).

[4] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, Numerical Recipes in Fortran 77, 2nd ed., Cambridge University Press (1992).

[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.